

ASSIGNMENT-20.2

NAME: HABEEBA KHANAM

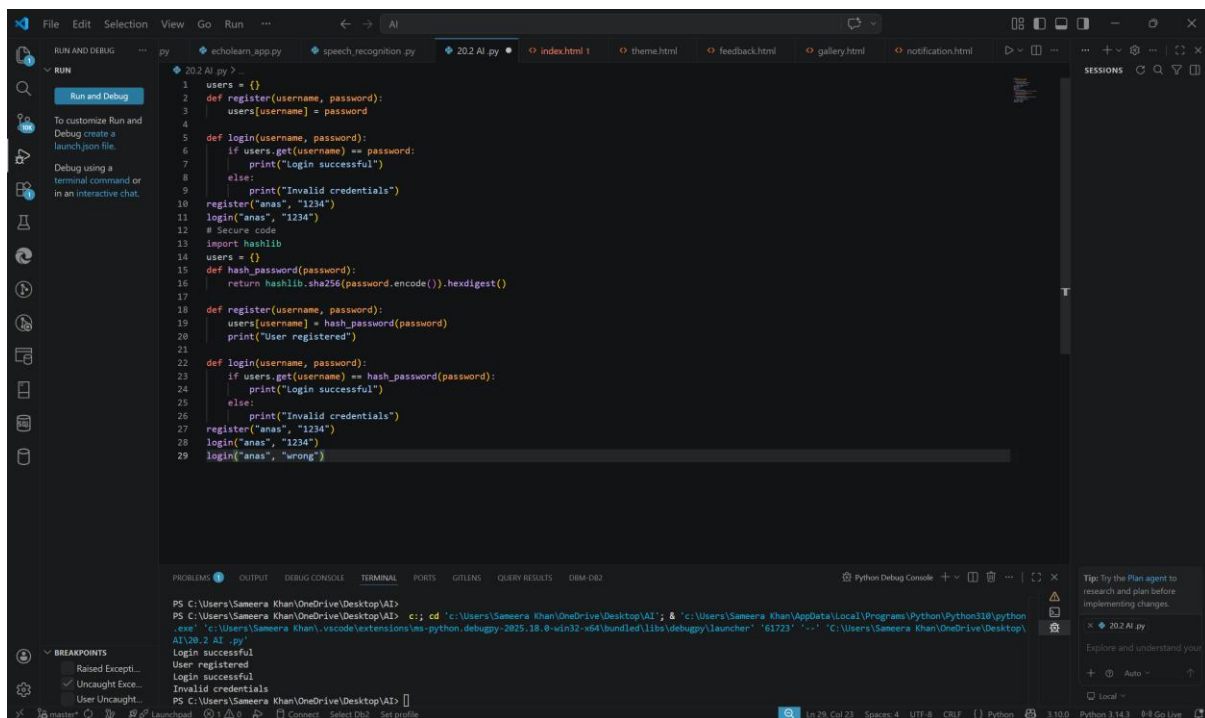
HALL TICKET :2303A51474

BATCH NO:29

LAB 20 – SECURITY TESTING: IDENTIFYING VULNERABILITIES IN AI-

TASK 1 – PASSWORD SECURITY ENHANCEMENT GENERATED CODE

CODE:



```
1 users = {}
2 def register(username, password):
3     users[username] = password
4
5 def login(username, password):
6     if users.get(username) == password:
7         print("Login successful")
8     else:
9         print("Invalid credentials")
10
11 register("anas", "1234")
12 login("anas", "1234")
13 # Secure code
14 import hashlib
15 users = {}
16 def hash_password(password):
17     return hashlib.sha256(password.encode()).hexdigest()
18
19 def register(username, password):
20     users[username] = hash_password(password)
21     print("User registered")
22
23 def login(username, password):
24     if users.get(username) == hash_password(password):
25         print("Login successful")
26     else:
27         print("Invalid credentials")
28
29 register("anas", "1234")
30 login("anas", "1234")
31 login("anas", "wrong")
```

Terminal Output:

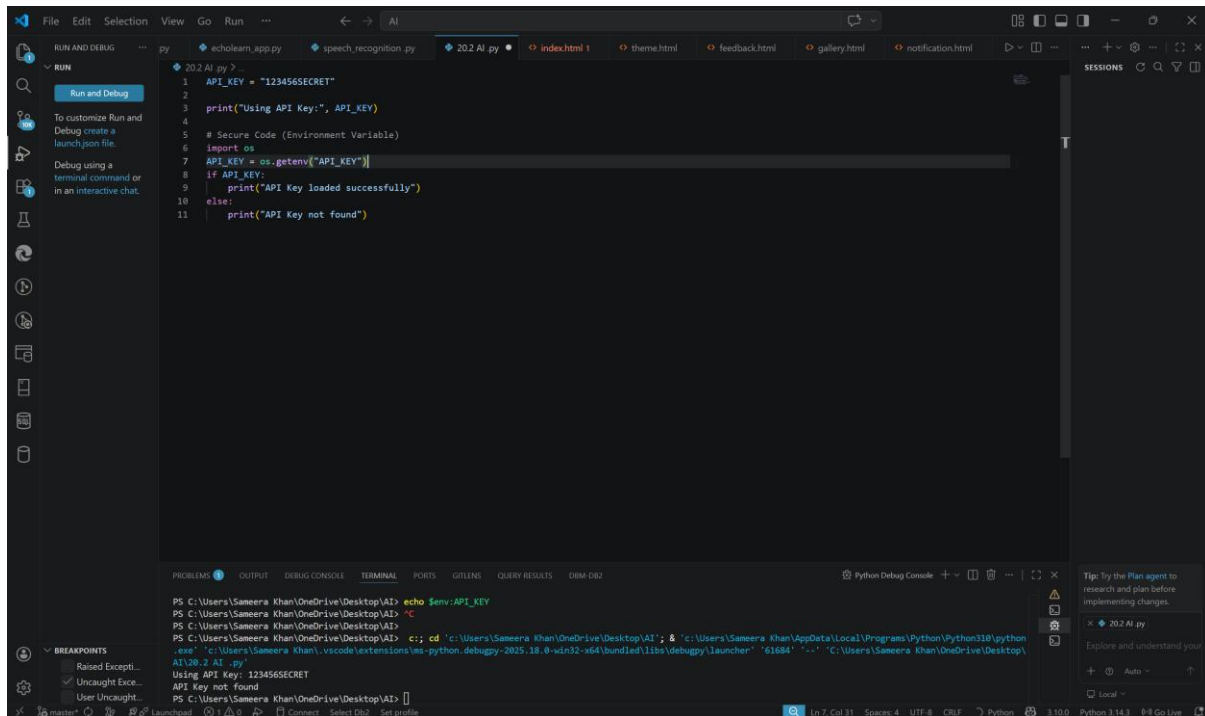
```
PS C:\Users\Sameera Khan\OneDrive\Desktop\AI>
PS C:\Users\Sameera Khan\OneDrive\Desktop\AI> c:; cd 'c:\Users\Sameera Khan\OneDrive\Desktop\AI'; & 'c:\Users\Sameera Khan\AppData\Local\Programs\Python\Python310\python.exe'; 'c:\Users\Sameera Khan\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '61723' '-' 'c:\Users\Sameera Khan\OneDrive\Desktop\AI\20.2 AI .py'
Login successful
User registered
Login successful
Invalid credentials
PS C:\Users\Sameera Khan\OneDrive\Desktop\AI>
```

OBSERVATION:

Passwords stored in plain text are vulnerable to theft if the database is compromised. Using hashing (SHA-256) secures passwords by storing irreversible encrypted values instead of actual passwords.

TASK 2 – SECURE API KEY MANAGEMENT

Code:



```
1 API_KEY = "123456SECRET"
2 print("Using API Key:", API_KEY)
3
4 # Secure Code (Environment Variable)
5 import os
6
7 API_KEY = os.getenv("API_KEY")
8
9 if API_KEY:
10     print("API Key loaded successfully")
11 else:
12     print("API Key not found")
```

Terminal Output:

```
PS C:\Users\Sameera Khan\OneDrive\Desktop\AI> echo $env:API_KEY
PS C:\Users\Sameera Khan\OneDrive\Desktop\AI>
PS C:\Users\Sameera Khan\OneDrive\Desktop\AI>
PS C:\Users\Sameera Khan\OneDrive\Desktop\AI> cd 'c:\Users\Sameera Khan\OneDrive\Desktop\AI' & 'c:\Users\Sameera Khan\AppData\Local\Programs\Python\Python310\python.exe' 'c:\Users\Sameera Khan\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '61684' '-' 'c:\Users\Sameera Khan\OneDrive\Desktop\AI\20_2 AI .py'
Using API Key: 123456SECRET
API Key not found
PS C:\Users\Sameera Khan\OneDrive\Desktop\AI>
```

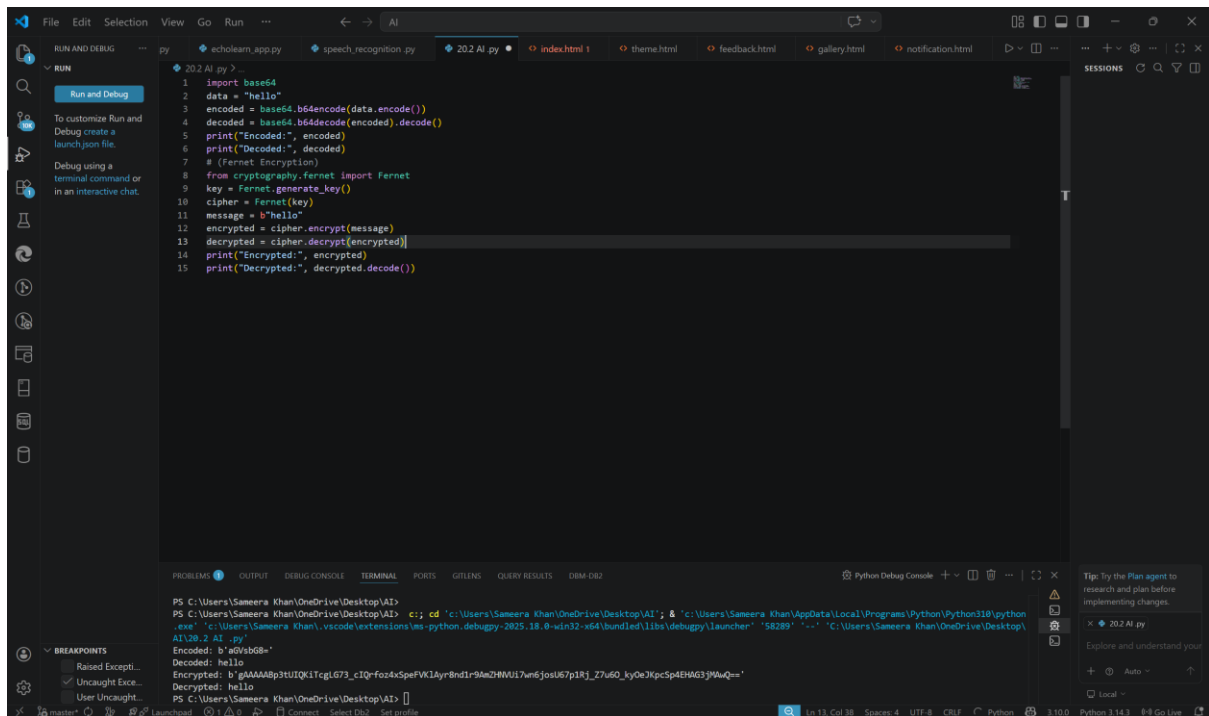
Observation:

Hardcoding API keys in source code exposes sensitive credentials to anyone with access to the code.

Using environment variables keeps API keys secure and prevents accidental leakage.

TASK 3 – WEAK ENCRYPTION DETECTION

CODE:



```
1 import base64
2 data = "hello"
3 encoded = base64.b64encode(data.encode())
4 decoded = base64.b64decode(encoded).decode()
5 print("Encoded:", encoded)
6 print("Decoded:", decoded)
7 # (Fernet Encryption)
8 from cryptography.fernet import Fernet
9 key = Fernet.generate_key()
10 cipher = Fernet(key)
11 message = b"hello"
12 encrypted = cipher.encrypt(message)
13 decrypted = cipher.decrypt(encrypted)
14 print("Encrypted:", encrypted)
15 print("Decrypted:", decrypted.decode())
```

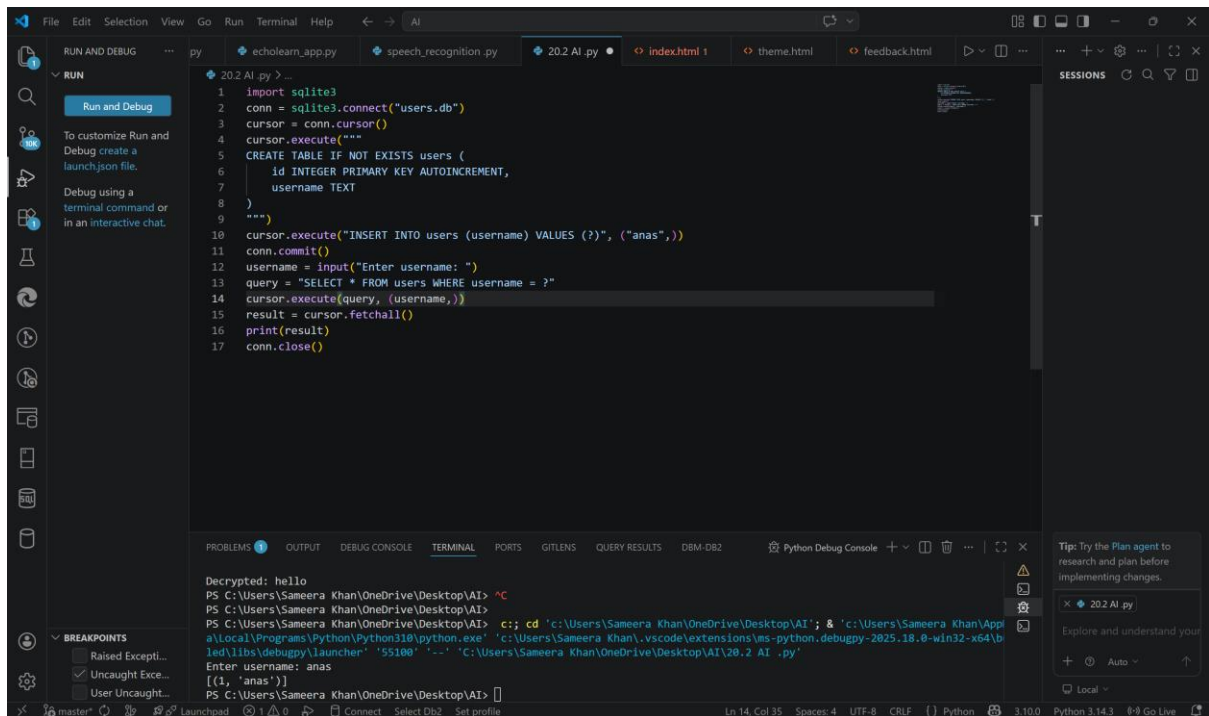
```
PS C:\Users\Sameera Khan\OneDrive\Desktop\AI>
PS C:\Users\Sameera Khan\OneDrive\Desktop\AI> cd 'c:\Users\Sameera Khan\OneDrive\Desktop\AI'; & 'c:\Users\Sameera Khan\AppData\Local\Programs\Python\Python310\python.exe' 'c:\Users\Sameera Khan\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '58289' '-' 'C:\Users\Sameera Khan\OneDrive\Desktop\AI\202 AI.py'
Encoded: b'bG90bG8='
Decoded: hello
Encrypted: b'gAAAAABp3tUIQk1TcgL673_cIQ-f0z4xSpeFWJ1AyR8ndI-94wZHWU17un6josuK7p1Rj_Z7u60_ky0eJkpcSp4EHAG3jMwQ=='
Decrypted: hello
PS C:\Users\Sameera Khan\OneDrive\Desktop\AI>
```

OBSERVATION:

Basic encoding methods like Base64 are not secure as they can be easily reversed.
Strong encryption (Fernet) ensures data confidentiality and requires a key for decryption.

TASK 4 – REAL-TIME APPLICATION: SECURITY REVIEW OF AI-GENERATED WEB SERVICE

CODE:



OBSERVATION:

Building SQL queries using user input directly leads to SQL injection vulnerabilities.
Using parameterized queries and validation prevents unauthorized database access.

TASK 5 – EXCEPTION HANDLING IMPROVEMENT

CODE:

```
File Edit Selection View Go Run Terminal Help
20.2 AI .py
1 a = int(input("Enter number: "))
2 b = int(input("Enter number: "))
3 print("Result:", a / b)
4 # Secure Code
5 try:
6     a = int(input("Enter first number: "))
7     b = int(input("Enter second number: "))
8     result = a / b
9     print("Result:", result)
10 except ZeroDivisionError:
11     print("Error: Cannot divide by zero")
12 except ValueError:
13     print("Error: Invalid input. Enter numbers only")

PS C:\Users\Sameera Khan\OneDrive\Desktop\AI> c:: cd 'c:\Users\Sameera Khan\OneDrive\Desktop\AI'; & 'c:\Users\Sameera Khan\AppData\Local\Programs\Python\Python310\python.exe' 'c:\Users\Sameera Khan\.vscode\extensions\ms-python,debugpy-2025.18.0-win32-x64\bin\debugpy_launcher' '60828' '--' 'C:\Users\Sameera Khan\OneDrive\Desktop\AI\20.2 AI .py'
Enter number: 10
Enter number: 0
Result: 5.0
Enter first number: 10
Enter second number: 0
Error: Cannot divide by zero
```

OBSERVATION:

Programs without exception handling may crash on invalid input or runtime errors.

Using try-except blocks improves reliability and provides user-friendly error messages.